

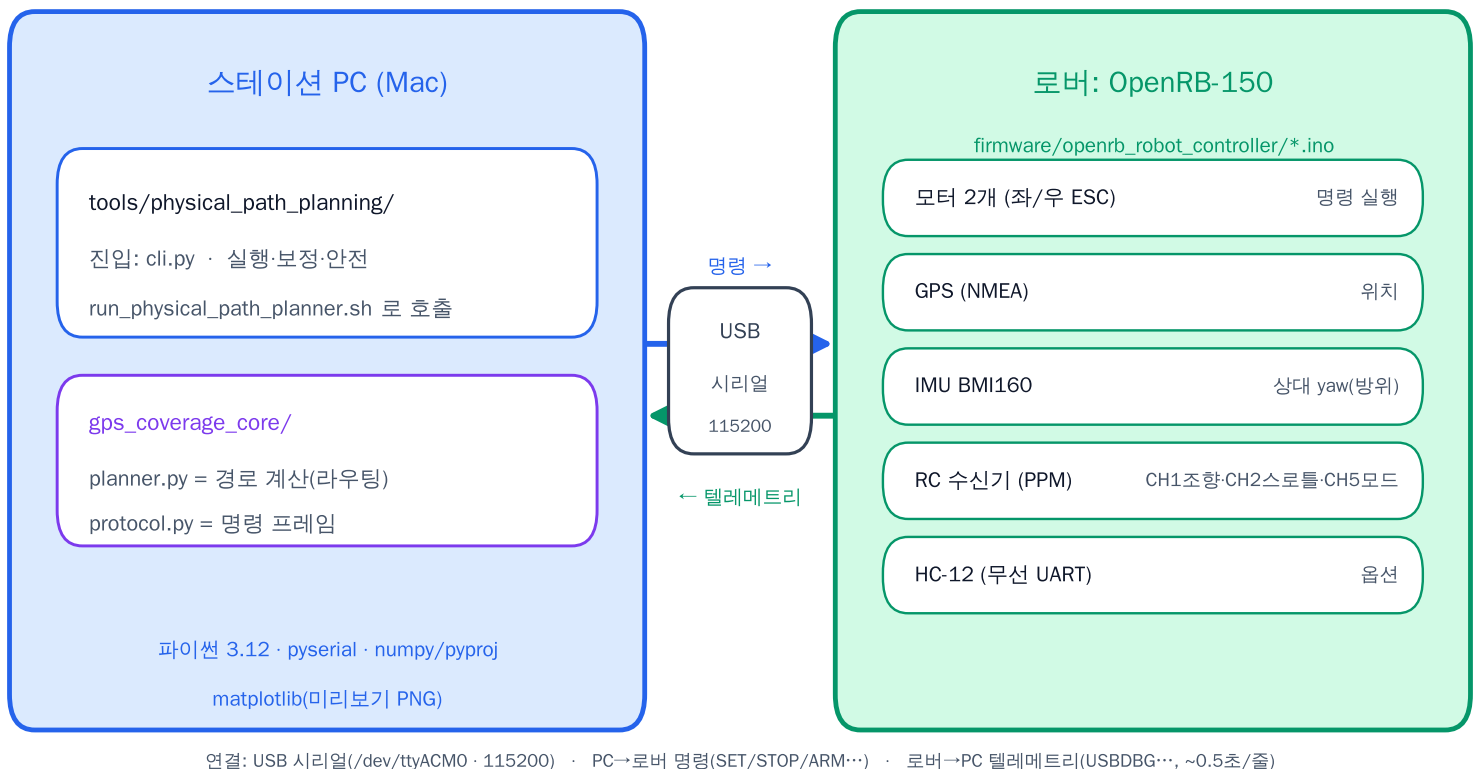
스테이션(PC) 라우팅 · 구동 코드 가이드

github.com/seonukkim/gps_hc12_robot · OpenRB-150 로버 + HC-12 + GPS/IMU

한 줄 요약

로버 코드는 .ino 하나. PC의 '라우팅 파이썬'은 tools/physical_path_planning/ (실행) + gps_coverage_core/planner.py (경로 계산). 둘은 USB 시리얼 한 줄로 연결된다.

전체 구조 — 누가 무엇을 실행하고 어떻게 연결되나



역할 분리

두뇌(경로·판단) = 파이썬 | 안전 실행(모터·센서) = 펌웨어(.ino)

① 유선 — 두뇌가 PC

PC가 실시간으로 저수준 구동 명령 전송.
모드: auto-relative-run / run / execute-plan
USB를 뽑으면 데드맨이 즉시 정지(설계).
→ 상대가 말한 'PC 라우팅 코드'는 보통 이것

② 무선 — 두뇌가 로버

PC로 온보드 패턴 펌웨어를 1회 업로드만.
이후 USB 없이 RC 송신기만으로 동작.
모드: rc-auto-pattern (ㄹ자 왕복)
→ PC 파이썬은 빌드·업로드 도구로만 사용

안전 원칙: 모든 실행 요약에 ready_for_full_path_following=false 강제 (checks.py)

1) 설치 (PC, 최초 1회)

```
uv sync --extra dev          # 또는 pip install -r requirements.txt
```

펌웨어 업로드 모드는 arduino-cli + OpenRB-150:samd:OpenRB-150 보드 코어 필요

2) 모든 실행의 공통 진입점

```
bash scripts/run_physical_path_planner.sh <모드> [옵션]
```

얕은 디스패처 → 실제 로직은 tools/physical_path_planning/cli.py 의 cmd_* 함수

주요 모드

모드	모터	역할
diagnose	×	연결·센서 확인(텔레메트리만 요약)
gps-wait	×	쓸 만한 GPS 픽스 대기
rc-input-diagnose	×	RC 수신기 채널 입력 진단
manual-control	RC	PPM 수동 조작 펌웨어 업로드·모니터
preview	×	꺾자 커버리지 경로 계획 + PNG 렌더
inspect-plan	×	저장된 플랜/이미지 확인
tune-motion	●	대화형 모터 캘리브레이션
calibrate-turn	●	회전 각도 캘리브레이션(IMU yaw)
calibration-check	×	캘리브레이션 완성도 점검
align-heading	●	첫 레인 방향으로 로버 정렬
run / execute-plan	●	경로를 우선으로 실행
auto-relative-run	●	AUTO 스위치 대기 후 상대경로 1회 우선 실행
rc-auto-pattern	●	무선 온보드 꺾자 패턴 펌웨어 업로드

● = 모터 구동 · RC = 수동 조작 · × = 모터 안 움직임

산출물(트레이스 CSV·요약 JSON·PNG)은 outputs/ 아래에 저장(깃 커밋 안 함).

geometry · calibration · telemetry · safety · checks → executor → controller → cli

tools/physical_path_planning/ — PC 실행/구동의 본체

cli.py	사용자 CLI·시리얼 오픈·펌웨어 빌드/업로드·요약(JSON) 기록 (가장 큼)
geometry.py	르자 경로 기하 생성. 코너를 회전→스텝→회전으로 분해
calibration.py	전진/후진/회전 캘리브레이션 값·회전 실제각(target_angle_deg)
telemetry.py	'USBDBG ...' 라인 파싱 + GPS/IMU/RC/모터 필드 접근자
executor.py	시리얼에 명령 쓰기 · 가드 펄스 FSM(ARM→ACK→완료→STOP)
controller.py	경로 실행 감독 루프(stop_correct_go): 구동→정지→읽기→보정 반복
alignment.py	초기 헤딩 정렬(GPS 변위 프로브 + IMU 제자리 회전)
tuning.py	대화형 캘리브레이션 후보 조정·승인·백업/리셋
preview.py	계획 경로를 PNG로 렌더(현장에서 눈으로 확인)
safety.py / checks.py	ACK/STOP·출력0 확인 · ready_for_full_path_following=false 강제

gps_coverage_core/

순수 경로 계산(하드웨어 비의존)

planner.py ★

위경도↔미터, 레인 오프셋, 왕복 웨이포인트

protocol.py

PC↔로버 명령 프레임·XOR 체크섬·클램프

geo.py

거리·방위 계산

nmea.py

GPS NMEA 파싱

imu.py

IMU 계산 헬퍼

side_tool_planner.py

보조 도구 경로

firmware/.../openrb_robot_controller.ino

로버의 유일한 컨트롤러. 하나의 파일을

컴파일 -D 플래그로 여러 모드로 빌드.

PPM 디코드·GPS·IMU·모터·안전게이트

USBDBG 텔레메트리 회신

ros2_ws/ (현재 스켈레톤, 미사용)

coverage_planner / hc12_bridge /

station_mission / waypoint_follower 노드

→ 지금 전달할 라우팅 코드엔 불포함

firmware/*_probe, *_test 폴더는 GPS/IMU/PPM 배선 점검용 일회성 진단 스케치(운용과 무관).

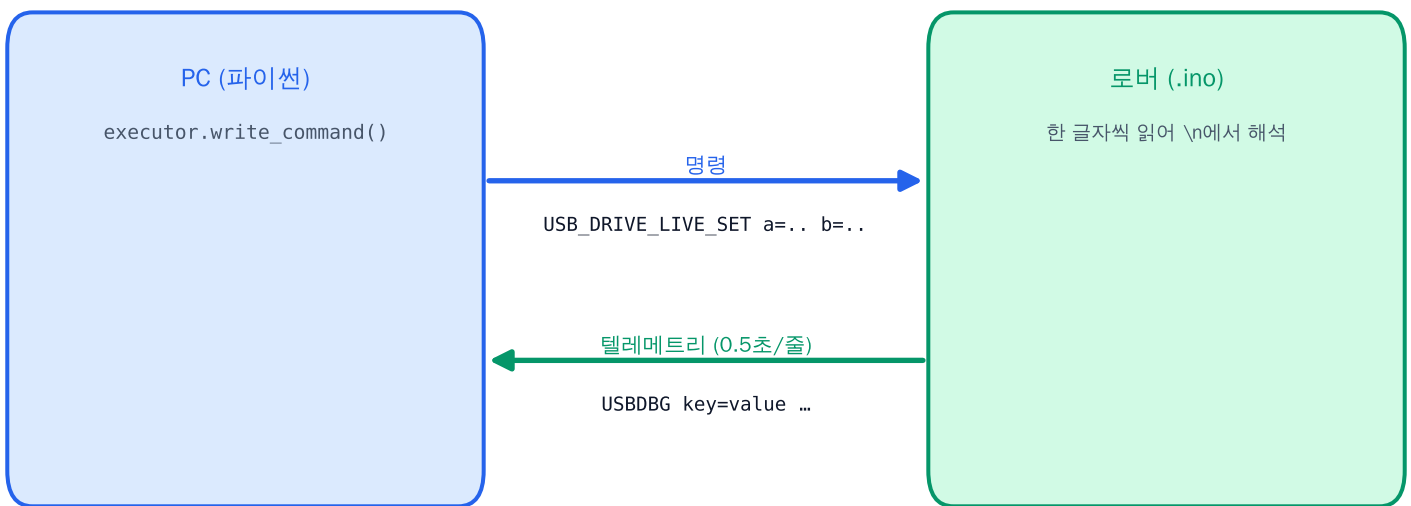
① 빌드타임 — 파이썬이 펌웨어를 빌드·업로드

cli.py → `arduino-cli compile/upload` (같은 .ino, 모드별 -D 플래그)

예) 무선 패턴: `-DRC_AUTO_PATTERN=1 -DMANUAL_CONTROL_PPM=1 -DIMU_ENABLE=1 ...`

‘어떤 파이썬 모드로 올렸나’ = ‘로버가 어떤 펌웨어 모드로 도나’

② 런타임 — USB 시리얼 문자열 (/dev/ttyACM0 · 115200)



PC → 로버 (개행 종료 ASCII)

`USB_DRIVE_LIVE_SET seq=N a=<전후진> b=<조향> ms=.. ttl=..` 연속 구동 setpoint(핵심)

`USB_DRIVE_LIVE_STOP seq=N` 연속 구동 정지

`USB_PULSE_TEST_ARM/..._STOP seq=N` 가드 펄스 1회

`STOP` 즉시 정지

a = physical A(전/후진), b = physical B(좌/우 조향), 각각 [-1,1] 클램프 → 좌우 바퀴로 믹싱

로버 → PC (USBDBG 한 줄, 공백 구분 key=value)

`USBDBG mode=AUTO_RUNNING event=ACK rc_ok=true auto_sw=true`

`gps_sats=7 gps_hdop=1.2 imu_relative_yaw_deg=172.8`

`final_left_cmd=0.31 final_right_cmd=0.29 physical_output_active=true ...`

`telemetry.parse_usbdbg_rows()`로 dict 파싱 · event=ARM/ACK/STOP는 가드 펄스 상태 확인용

무선으로 쓰려면 위 대신 rc-auto-pattern 으로 1회 업로드 후 USB를 뺐고 RC 송신기로 운용

1

연결·센서 확인 (모터 X)

· USBDBG가 올라오고 GPS/IMU/RC 필드가 보이는지 확인.

```
bash scripts/run_physical_path_planner.sh diagnose \
  --out-dir outputs/physical_path_planning/diagnose
```

2

GPS 픽스 대기 (모터 X)

· gps_sats ≥ 5 · hdop가 임계값 이하로 안정될 때까지.

```
bash scripts/run_physical_path_planner.sh gps-wait \
  --timeout-s 300 --min-sats 5 --max-hdop 2.5 \
  --out-dir outputs/.../gps_wait
```

3

경로 미리보기 → PNG (모터 X)

· 경로를 눈으로 먼저 확인(preview_*.png).

```
bash scripts/run_physical_path_planner.sh preview \
  --goal-mode relative_enu --goal-east-m 1.8 --goal-north-m 1.8 \
  --workspace-width-m 1.8 --step-spacing-m 0.6 --out-dir outputs/.../preview
```

4

(필요 시) 모터 캘리브레이션

· 전진/후진/회전 각각. 회전은 calibrate-turn.

```
bash scripts/run_physical_path_planner.sh tune-motion \
  --primitive forward --out-dir outputs/.../tune_forward
```

5

첫 레인 방향 정렬

· GPS 변위 프로브 + IMU 제자리 회전으로 정렬.

```
bash scripts/run_physical_path_planner.sh align-heading \
  --out-dir outputs/.../align
```

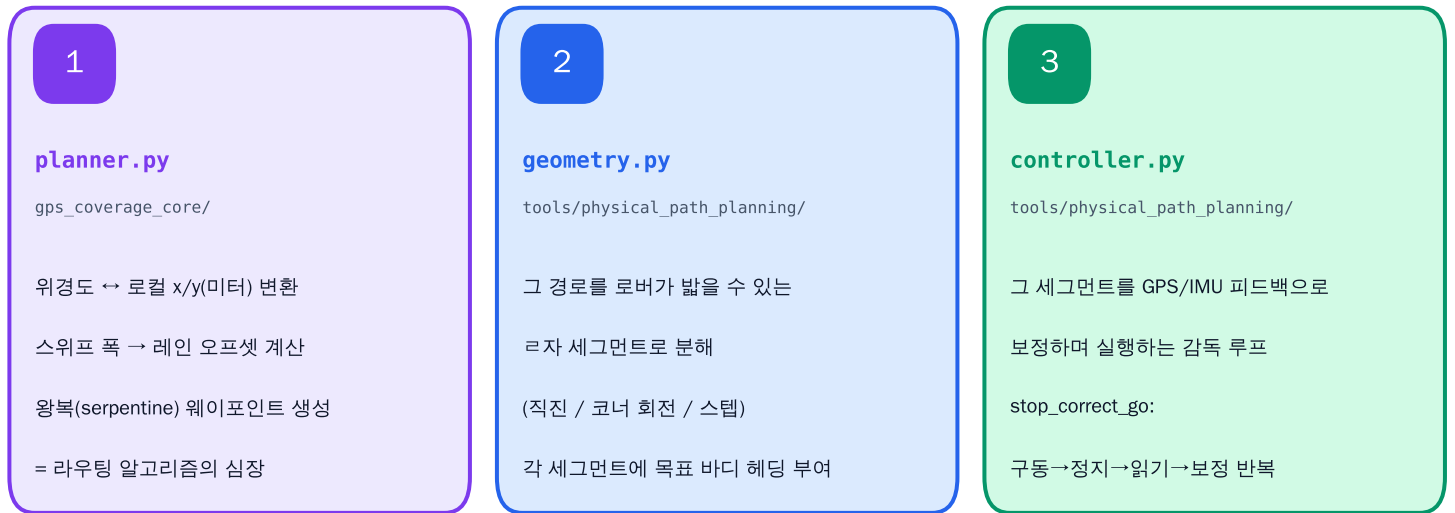
6

유선 실행 — 상대경로 1회

· RC를 AUTO로 넘기면 현재 GPS를 원점으로 시작.

```
bash scripts/run_physical_path_planner.sh auto-relative-run \
  --goal-east-m 1.8 --goal-north-m 1.8 \
  --workspace-width-m 1.8 --step-spacing-m 0.6 --out-dir outputs/.../auto_run
```

A. 순수 ‘경로 계산 로직’을 원하면 → 이 3파일 (우선순위 순)



B. ‘PC가 로버를 어떻게 움직이나(구동 인터페이스)’를 원하면 → executor.py + gps_coverage_core/protocol.py

데이터 흐름 요약



참고 문서 (docs/)

README_physical_path_planning.md	현장 실행 런북(유선/무선)
physical_path_planning_architecture.md	모듈 아키텍처
station_routing_code_guide.md	이 PDF의 원본 텍스트(전체 상세)
claude_collaboration_guide.md	제어 의미론·트러블슈팅 결정 트리
protocol.md · wiring.md · rc_channel_map.md	프로토콜·배선·RC 채널

안전: 이 스택은 완전 자율이 아니라 감속·가드 구동. 실제 주행 전 preview로 경로 확인, GPS 픽스·캘리브레이션 먼저.